PROJECT

On

AI Knowledge Base Chatbot

Submitted By:

Shaik Irfan

Shaikirfan7025@gmail.com

+91- 8328460949

Github Repo:[LINK]

Abstract:

This project presents an AI-powered Knowledge Base Chatbot built using Retrieval-Augmented Generation (RAG). The system allows users to upload documents and ask questions based on the content. It uses Qdrant as a vector database to store embeddings, and a language model to generate answers. The frontend is built using Streamlit, and the entire system is deployed using Docker containers. The chatbot improves information retrieval and provides accurate, context-based responses.

Objective:

- To build an intelligent chatbot using RAG architecture
- To store document embeddings using Qdrant vector database
- To retrieve relevant context for answering queries
- To create an interactive UI using Streamlit
- To deploy the system using Docker

Technology Used:

- Python
- Streamlit
- LangChain
- Qdrant
- HuggingFace Transformers
- Docker

Proposed Methodology:

The methodology of the proposed AI Knowledge Base Chatbot is based on the Retrieval-Augmented Generation (RAG) architecture. The system is designed to retrieve relevant information from user-provided documents and generate accurate answers using a language model.

1. Document Upload: The system allows users to upload documents in PDF or TXT format through the Streamlit interface. These documents serve as the knowledge base for the chatbot.

2. Text Extraction: Once the document is uploaded, the content is extracted. For PDF files, a PDF loader is used to read and extract text from each page. For text files, the content is directly read from the file.

3. Text Chunking: The extracted text is divided into smaller chunks using a text splitter. This is done to improve retrieval accuracy and ensure efficient processing. Each chunk contains a manageable portion of the text with slight overlap to preserve context.

4. Embedding Generation: Each text chunk is converted into a numerical representation (embedding) using a pre-trained sentence transformer model. These embeddings capture the semantic meaning of the text, allowing similarity-based search.

5. Storing in Vector Database (Qdrant): The generated embeddings are stored in Qdrant, a vector database. Each chunk is stored along with its embedding, enabling fast similarity search during query processing.

6. User Query Input: The user enters a question in the chatbot interface. This query is also converted into an embedding using the same embedding model.

7. Retrieval of Relevant Chunks: The system searches the Qdrant database to find the most relevant text chunks based on similarity between the query embedding and stored embeddings. The top matching chunks are selected as context.

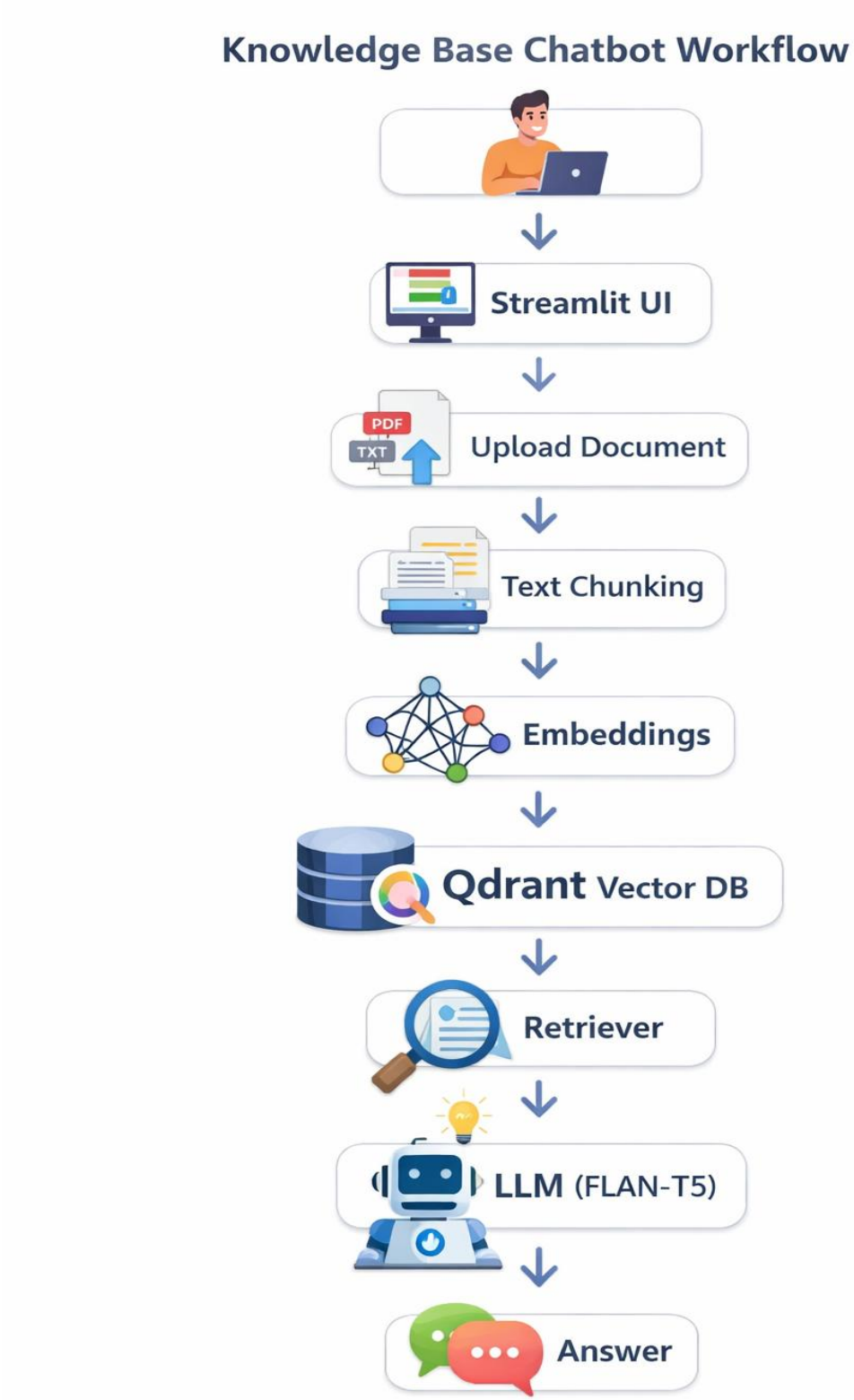
8. Context-Based Answer Generation: The retrieved context is passed to a language model (FLAN-T5). The model generates a response based on the provided context and the user's question.

9. Displaying the Answer: The generated answer is displayed to the user in the Streamlit interface. Additionally, the system can show the source text used for generating the answer, improving transparency.

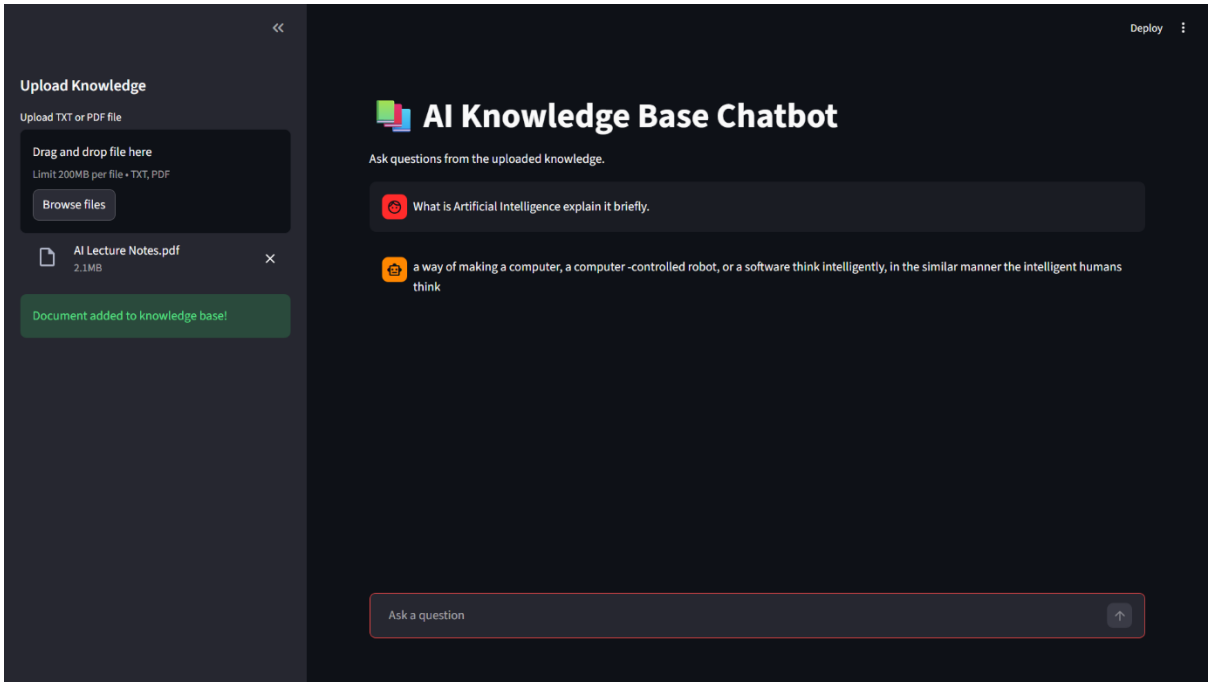
10. Deployment using Docker: The entire system is containerized using Docker. Separate containers are used for the Qdrant database and the chatbot application.

This methodology ensures efficient document processing, accurate information retrieval, and meaningful response generation using AI techniques.

Block Diagram



Results



Conclusion

The project successfully demonstrates the implementation of an AI-powered Knowledge Base Chatbot using modern technologies such as LangChain, Qdrant, Hugging Face models, and Streamlit. By combining retrieval and generation techniques, the system overcomes the limitations of traditional chatbots that rely only on predefined responses. The use of embeddings and vector databases enables efficient and accurate information retrieval from large documents. The project also highlights the importance of scalable deployment using Docker and modular design for real-world applications.

However, the system has certain limitations, such as slower response time on low-resource systems and dependency on model performance. These can be improved in future work by integrating lightweight models like LLaMA via Ollama and optimizing retrieval mechanisms.

In conclusion, the developed chatbot provides a practical and efficient solution for document-based question answering and has potential applications in education, customer support, and enterprise knowledge management systems.